

Spike: Task 24**Title:** Measuring Performance**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate my ability to test performance and measure optimising the code while using SDL2 library.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\24 - Spike - Profiling, Performance and Optimisation\24 Performance\24 Performance

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- SDL2 Library

Tasks undertaken:

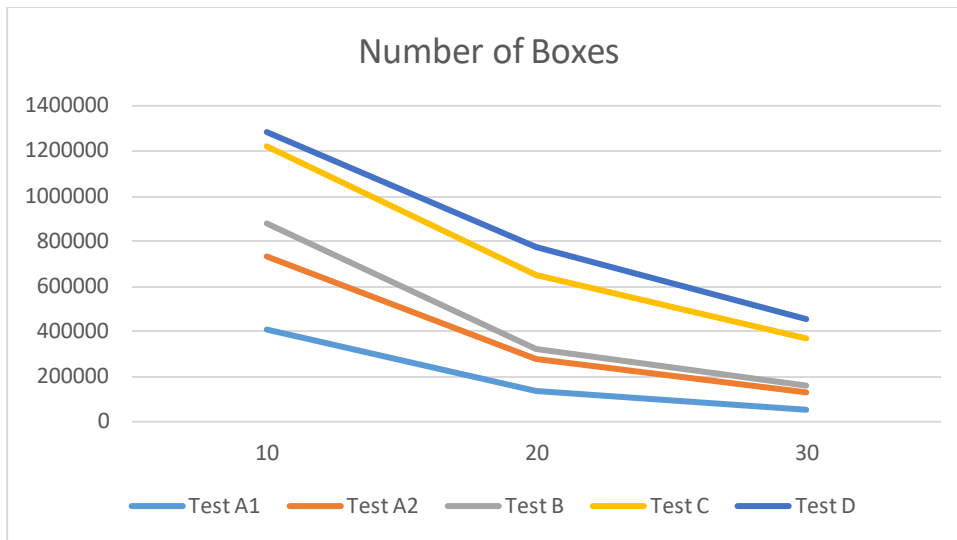
- Project setup for SDL2 library dependencies
- Using sample code that is provided.

Collision Tests:

1. Test A (crash_test_A): This test checks for collisions using box indices. For two boxes, A and B, it calculates the four sides of each rectangle (top, bottom, left, and right). The logic is based on the principle that for two rectangles to not be colliding, one's top is below the other's bottom, one's bottom is above the other's top, one's right is to the left of the other's left, or one's left is to the right of the other's right. If none of these conditions are met, then the rectangles are colliding. The method accesses the global array of boxes using indices to retrieve the position and size data for each box.

2. Test B (crash_test_B): This test is structurally like Test A, but instead of accessing boxes via indices, it directly takes two CrashBox structures as arguments. This means the box structures are passed by value, implying that they are copied for each function call. The collision determination is the same as in Test A, but this test evaluates the performance impact of passing structures by value versus accessing them via an array.
3. Test C (crash_test_C): Test C is a slight modification of Test B. Instead of passing CrashBox structures by value, it passes them by reference. This means that instead of copying the structures when calling the function, references to the original boxes are used. By avoiding copying large structures, performance is expected to be improved, especially as the size of the structures or the frequency of function calls increases.
4. Test D (crash_test_D): This test introduces a more concise collision check. It still checks the top, bottom, left, and right boundaries of two boxes, but does so using fewer lines of code. It aims to provide an optimized way to determine if two boxes are colliding by directly using the box's dimensions and positions in the collision determination logic, potentially improving readability and performance.

Box Count	Test A1 Loops	Test A2 Loops	Test B Loops	Test C Loops	Test D Loops
10	408634	732551	878941	1220718	1284011
20	134623	277841	322479	653769	773811
30	53046	130274	160165	369503	455086



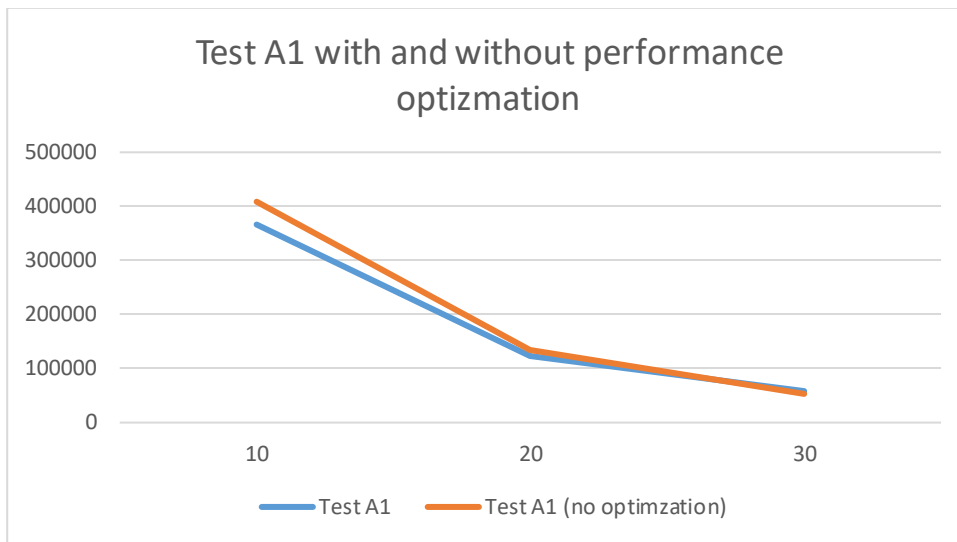
These recorded results are from the initial sample code provided without any optimisation and render turned to false where I only change BOX_COUNT to 10 and increase it by 10 for each executed run and therefore, as you can see on the chart the number of loops for a test time of 3 seconds keeps on decreases as I keep increasing the box count.

Collision Tests with Performance Optimization:

When optimizing the code to test performance I used the inclusion of the chrono library to help with precise time measuring, reduced collision detection calls as the original code compared call boxes when checking for collision while in my version, we don't compare boxes to itself or any boxes as it already has compared with.

To make the test runs more flexible, a function pointer **crash_test_all_ptr** is introduced which can be dynamically set to any of the collision detection algorithms. We also use cached rectangle in the crashbox struct that includes **SDL_Rect r** to reduce the need for continuous recalculations and this helps with rendering. Finally, to test the framework performance we improved the **run_test** function to run a test for specified test time, count the number of iterations it completes. This helps to compare the efficiency of the various collision detection algorithms.

Box Count	Test A1 Loops	Test A2 Loops	Test B Loops	Test C Loops	Test D Loops
10	366336	707274	812777	1112622	1169232
20	123646	267056	338873	618124	729602
30	58219	129947	171328	361757	444952



Based on the results when comparing the unoptimized and optimized Test A1 loop counts, it clearly displays a reduction of 10.35% and 8.16% for the 10 and 20 box count tests, respectively, indicating successful optimization for these box counts. However, for the 30-box scenario, the loop count unexpectedly increased by 9.75% and similar to the unoptimized result for 30 box counts. This suggests that the optimization improved efficiency for smaller data sizes but may have introduced inefficiencies or was less beneficial for larger data sizes. A deeper investigation into the optimization's nature and potential specific-case overheads would be necessary to fully understand this behaviour.